Università degli Studi di Milano Bicocca

Scuola di Scienze

Dipartimento di Informatica, Sistemistica e Comunicazione

Corso di laurea in Informatica

Theoretical Foundations of Large Language Models: Text Prediction and Limitations

Relatore: *Guido Giuseppe Fiorino*

Relazione della prova finale di:

Huadong Liao

Matricola 886110

Anno Accademico 2025-2026

Preface

During my internship, I had the opportunity to work with various tools based on artificial intelligence, which not only amazed me with what they could contribute in a work context, but also reinforced a curiosity I already had in late 2022 (when ChatGPT was launched): how large language models generate text. This led me to explore the mechanisms behind this process.

Abstract

This work provides an overview of the theoretical foundations that enable LLMs to generate text. It first introduces the evolution from classical artificial intelligence to modern neural networks, covering topics such as the perceptron, multi-layer neural networks, activation functions, and the training process through backpropagation and gradient descent.

The work then examines how large language models are trained, focusing on next-token prediction, mini-batch training, and optimization techniques such as stochastic gradient descent and the AdamW optimizer. It then discusses embeddings and semantic similarity metrics, which allow words and concepts to be represented in high-dimensional vector spaces and are particularly important for language models. It also introduces the distinction between static and contextual embeddings.

The architecture of modern transformer-based models is then analyzed, including the motivations behind the introduction of the transformer architecture, positional encoding, self-attention, multi-head attention, and the decoder-only transformer structure used in most modern LLMs.

Finally, the work discusses several key limitations of large language models, including the grounding problem, hallucinations, context window length constraints, and the challenges associated with training these systems.

Contents

1 Introduction **6**
1.1 Definition of Artificial Intelligence 7
1.2 What Is a Large Language Model 8

2 Neural Networks **9**
2.1 From Symbolic AI to Machine Learning 10
2.2 The Perceptron 12
2.2.1 Limitations of a Single Perceptron 14
2.3 Activation Functions 15
2.4 Multi-Layer Perceptron 16
2.5 Training Neural Networks 19
2.5.1 Feedforward 19
2.5.2 Loss Function 19
2.5.3 Backpropagation 20
2.5.4 Gradient Descent 23

3 Training Large Language Models **24**
3.1 GPT-3 Parameters Overview 24
3.2 Training Data Composition 25
3.2.1 Tokenization 25
3.3 The Pre-Training Phase 27
3.3.1 Next-Token Prediction 27
3.3.2 Mini-Batch Training 28
3.3.3 Advantages of Mini-Batch Training 29
3.3.4 Softmax and Cross-Entropy Loss 30
3.3.5 Gradient with Respect to the Logits 31
3.3.6 Mini-Batch Stochastic Gradient Descent 32
3.3.7 The AdamW Optimizer 33
3.4 Supervised Fine-Tuning 34
3.5 Alignment Phase 35
3.5.1 Reward Models and PPO 35
3.5.2 Direct Preference Optimization 36

4 Embeddings **38**
4.1 What is an Embedding 38
4.2 Semantic Similarity Metrics 39

4.2.1 Cosine Similarity 39
4.2.2 Dot Product 40
4.3 Static Embeddings 41
4.3.1 Word2Vec 41
4.3.2 Linear Relationships in Static Embeddings 42
4.3.3 Limitations of Static Embeddings 43
4.4 Contextual Embeddings 43

5 Transformers **44**
5.1 Recurrent Neural Networks 45
5.2 From RNNs to Transformers 47
5.3 The Transformer Architecture 48
5.3.1 Positional Encoding 49
5.3.2 Self-Attention Mechanism 50
5.3.3 Multi-Head Attention 53
5.3.4 Masked Self-Attention 54
5.3.5 Feed-Forward Network 55
5.3.6 Residual Connection and Layer Normalization 56
5.3.7 Transformer Blocks (Decoder-only) 57
5.3.8 Output Layer 58

6 Limitations of Large Language Models **60**
6.1 The Grounding Problem 61
6.2 Hallucinations 62
6.3 Context Window Length 63
6.4 Controllability and Computational Cost 63

7 Conclusion **64**

References **65**

1 Introduction

In the last few years, Large Language Models have become the symbolic representation of modern artificial intelligence. When systems based on LLMs first became available to the public, they were often perceived as little more than simple chatbots. Early versions could produce surprisingly fluent text, but their answers were often inconsistent or incorrect. Despite this, their capabilities have improved rapidly. Today, they are integrated into many products and services and are increasingly used in professional and business contexts.

Because of the wide range of questions they can answer, these systems can give the impression of having almost unlimited knowledge and reasoning capabilities. Modern models can perform surprisingly well on tasks that require very advanced academic knowledge. One example is Humanity's Last Exam [1], a benchmark consisting of extremely difficult questions across various academic disciplines that typically require top-level human expertise. On this benchmark, models such as Gemini 3.1 Pro achieved a score of about 44.4% [2], illustrating their ability to solve problems that even the smartest humans struggle with.

Despite how "magical" these systems may appear, LLMs are fundamentally a specific category of deep learning models. Most modern LLMs are based on the Transformer architecture, and they are trained on absurdly large amounts of data. During the pretraining phase, the model learns statistical patterns in language through a simple objective: predicting the next token in a sequence of text. This autoregressive training process, repeated many times over vast amounts of data, is what allows LLMs to generate coherent and contextually relevant text.

1.1 Definition of Artificial Intelligence

Artificial Intelligence (AI) is a field of computer science that develops systems capable of performing tasks that normally require human intelligence, such as learning from data, recognizing patterns, making decisions, or generating text.

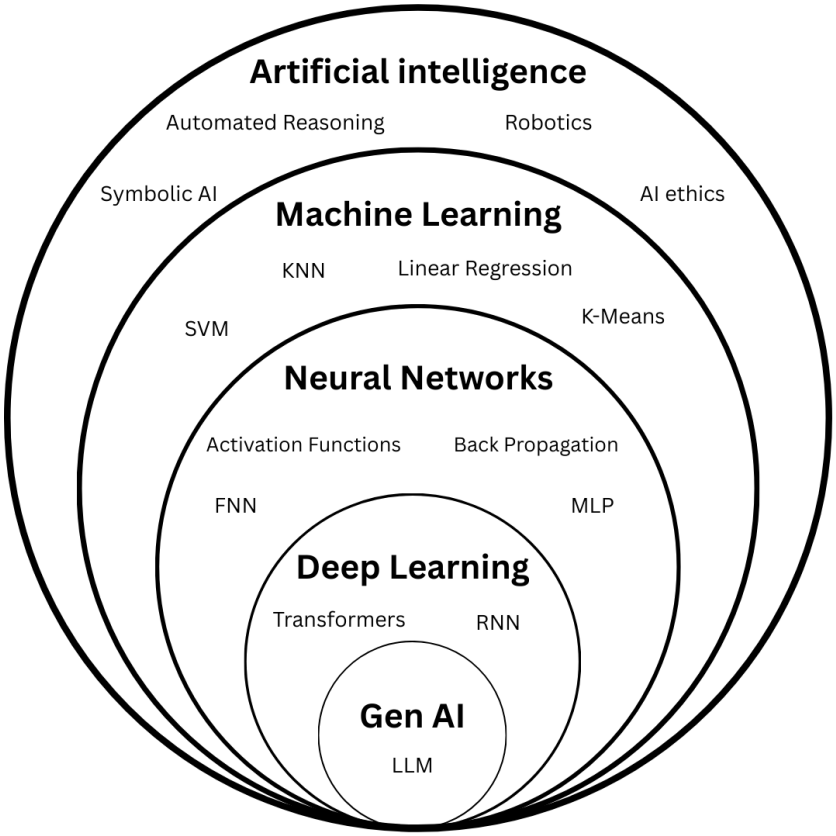

Figure 1: Hierarchical relationship between AI fields

1.2 What Is a Large Language Model

An LLM is a neural network based on the **transformer** architecture, which belongs to the family of deep learning models. Its main purpose is to generate coherent natural language given a sequence of text. It does not "understand" language in the human sense, but operates as a statistical system. Given a sequence of tokens $(x_1, \ldots, x_n)$, the model estimates a probability distribution over the entire vocabulary V for the next token w:

$$P(X_{n+1} = w \mid X_1 = x_1, \ldots, X_n = x_n), \quad x_1, \ldots, x_n \in V,\ w \in V$$

LLM platforms generate text gradually rather than all at once. This is not a stylistic choice, but a direct consequence of the autoregressive nature of the model.

The input sequence is converted into smaller units called *tokens* before being processed. In some cases, a token corresponds to a full word, but in other cases, it may represent a subword (for example "net" and "work" in "network"). Different language models use different tokenization algorithms, but the principle is similar: each model defines its own fixed vocabulary of tokens V, and every input text must be decomposed into elements of this vocabulary before being processed. During inference, the prompt provided by the user is first tokenized into a sequence $(x_1, \ldots, x_n)$. The model then predicts a probability distribution over all tokens in the vocabulary for the next position in the sequence. A token is then sampled from the probability distribution and appended to the sequence. This process is repeated until a special token EOS (end-of-sequence) is chosen.

The term "large" refers to the scale of the model: many parameters, massive training data, and huge computational requirements.

2 Neural Networks

Neural networks are statistical models designed to approximate complex relationships between inputs and outputs. They learn from data in order to predict outcomes with minimal error, such as classifying an image or estimating a house price from its features.

Large Language Models are very deep and complex neural networks. Fundamentally, they operate as mathematical functions that assign a probability to each token $w \in V$, given a sequence of tokens $x_1, \ldots, x_n$. The model then samples from this distribution to select the next token. During training, its parameters are adjusted to minimize prediction error, which allows it to generate coherent text.

Since LLMs belong to the broader deep learning family, standard neural network principles guide their training. Mechanisms such as internal parameter adjustments, non-linear activation functions, feedforward passes, backpropagation, and cross-entropy loss are applied as well. Therefore, before introducing specific mechanisms like embeddings and attention, it is useful to briefly review the fundamental concepts underlying neural networks.

2.1 From Symbolic AI to Machine Learning

Functions do not perfectly describe the world. That would be convenient, but reality is rarely that simple. At best, functions can approximate the patterns observed in data.

In early artificial intelligence, systems were often built by writing rules manually, an approach known as symbolic AI [3]. The idea was that, if the correct rule could be identified, every input would produce the correct output. However, this approach proved difficult to scale: real world phenomena are often too complex and irregular to be explained by explicit rules.

Consider a simple example: the points (x,y) = (2,1), (4,2), and (6,3) on the Cartesian plane. The pattern is obvious, and it is easy to infer that the next point would be (8,4). In this case, there exists a simple function that perfectly describes the relationship between input and output:

$$f(x) = \frac{1}{2}x$$

For every possible input x, the function produces the correct output.

Now consider a more realistic scenario where the observations are less regular, for example: (5,2), (8,3), (12,3), (18,5), (24,5). These points no longer lie perfectly on a single straight line, even though they roughly follow a linear trend. In theory, a function that passes exactly through all points may exist (a high-degree polynomial), but such a function would likely be unnecessarily complex and may capture irregularities (noise) specific to the observed data, a phenomenon known as **overfitting** in machine learning. Perfectly fitting the observed points does not guarantee that the same function will generalize well to unseen data.

For this reason, modern machine learning takes a different approach. Instead of trying to find the perfect rule, it aims to learn statistical patterns that generalize well while minimizing error on both seen and unseen data.

A well known approach that follows this paradigm is linear regression. It is one of the simplest machine learning models: rather than finding a function that passes exactly through every point, the model finds the line that best approximates the data by minimizing the overall error, typically measured using the mean squared error (MSE).

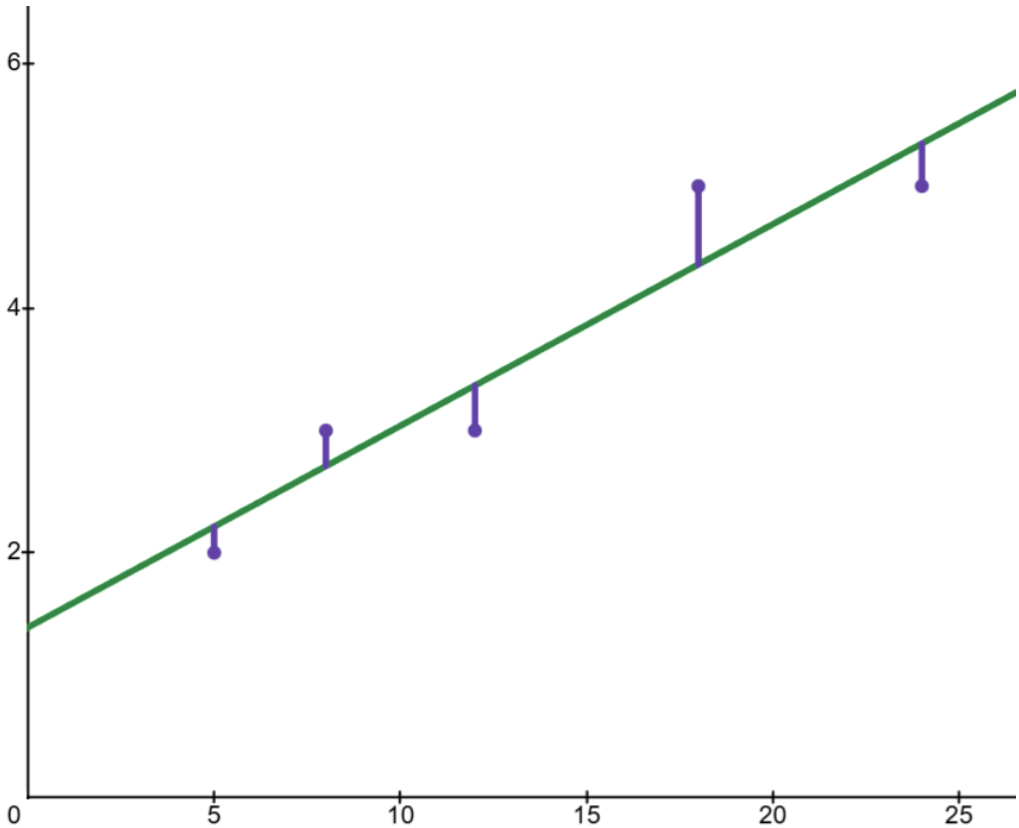

Figure 2: Linear Regression

2.2 The Perceptron

A perceptron is a simple mathematical model that computes a weighted combination of input variables and applies an activation function to produce an output.

Input variables x_i can be interpreted as features. For example, when choosing a restaurant, some possible features could be: Is it cheap? Are the meals good? Is there parking? Each of these characteristics corresponds to an input variable. The model associates a weight w_i to each input x_i. The weights represent how important each feature is for the decision. A feature with a larger weight influences the final decision more strongly than a feature with a smaller weight. The model computes a weighted sum of the inputs:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

where b is called bias, and acts as a threshold. Intuitively, if the weighted sum of the features exceeds the threshold, the model predicts a positive outcome (for instance, deciding to go to that restaurant).

The main purpose of the perceptron is to perform linear classification by drawing a decision boundary. In the previous example, the model is effectively classifying restaurants into two categories: those worth going to and those that are not.

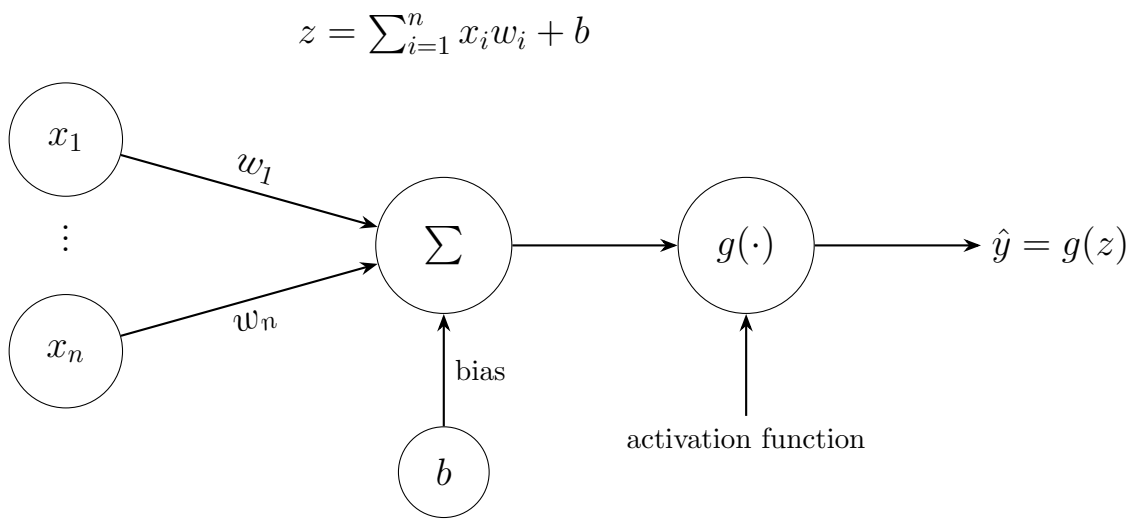

Figure 3: Structure of a perceptron

2.2.1 Limitations of a Single Perceptron

Despite its simplicity, a single perceptron has a key limitation: it can only represent linear decision boundaries. In other words, it can separate data only by drawing a straight line (or a hyperplane in higher dimensions).

Consider the simple case of a perceptron with two input features. In this setting, the model operates in a two-dimensional space, and the decision boundary corresponds to a straight line in the Cartesian plane. The perceptron attempts to position this line in a way that points belonging to different classes fall on opposite sides. However, not all patterns can be separated by a single straight line. As shown in Figure 4, the orange and green points cannot be correctly classified: no matter how the line is positioned, at least one point will always be misclassified.

To represent more complex patterns, the model must be able to construct non-linear decision boundaries. This is achieved by combining multiple perceptrons into layers with non-linear activation functions.

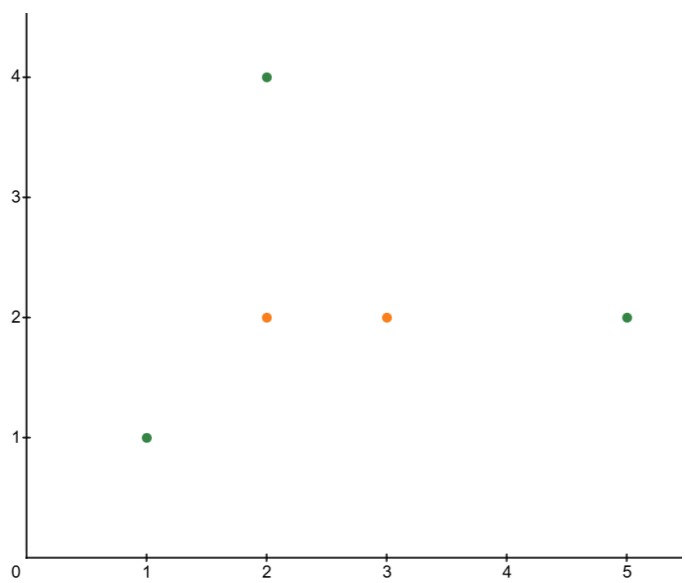

Figure 4: Non-linearly separable data in 2D

2.3 Activation Functions

Activation functions introduce non-linearity into neural networks. Without them, stacking multiple layers would still result in a linear transformation, limiting the model's ability to represent complex patterns.

Some commonly used activation functions include:

- **Step function**

$$f(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \qquad f : R \to \{0, 1\}$$

- **Sigmoid**

$$f(x) = \frac{1}{1 + e^{-x}} \qquad f : R \to (0, 1)$$

- **Hyperbolic tangent**

$$f(x) = \tanh(x) \qquad f : R \to (-1, 1)$$

- **Rectified Linear Unit (ReLU)**

$$f(x) = \max(0, x) \qquad f : R \to [0, +\infty)$$

- **Softmax**

$$f(\vec{x})_i = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}, \qquad i = 1, \ldots, n$$

$$f : R^n \to \left\{ \vec{y} \in (0, 1)^n \;\middle|\; \sum_{i=1}^{n} y_i = 1 \right\}, \qquad n \geq 2$$

2.4 Multi-Layer Perceptron

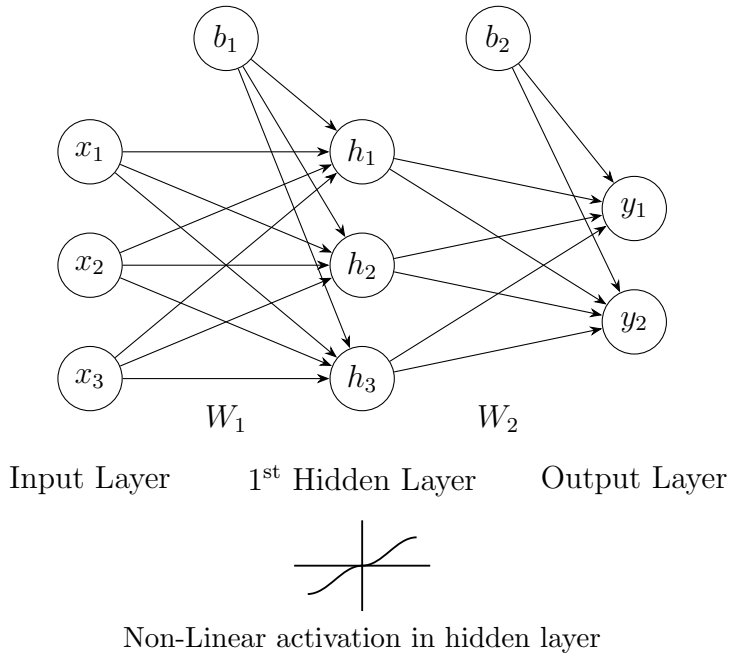

Figure 5: Multilayer perceptron

A single perceptron applies an activation function to a weighted sum of the inputs. The pre-activation of one neuron can be written as:

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b$$

and its output is:

$$h = g(z) = g(w_1x_1 + w_2x_2 + w_3x_3 + b)$$

where $g(\cdot)$ is a non-linear activation function.

Even if g is non-linear, a single perceptron can only produce a linear decision boundary in the input space. To see this, consider binary classification with a threshold t:

$$h \geq t$$

that is,

$$g(w_1x_1 + w_2x_2 + w_3x_3 + b) \geq t$$

If g is monotonic, this is equivalent to

$$w_1x_1 + w_2x_2 + w_3x_3 + b \geq c$$

for some constant $c = g^{-1}(t)$. This equation defines a hyperplane in the input space. Therefore, a single perceptron, despite including a non-linear activation, can represent only linear decision boundaries.

The situation changes when perceptrons are organized into multiple layers. In the Multi-Layer Perceptron shown in Figure 5, the hidden layer is computed as:

$$\mathbf{h} = g(W_1\mathbf{x} + \mathbf{b}_1)$$

where

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \qquad \mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}$$

The output layer is then

$$\mathbf{y} = W_2\mathbf{h} + \mathbf{b}_2$$

and by substitution,

$$\mathbf{y} = W_2\, g(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$$

This expression is no longer reducible to the form:

$$A\mathbf{x} + \mathbf{c}$$

For this reason, non-linear decision boundaries in neural networks arise from combining multiple layers of perceptrons with non-linear activations, allowing it to construct complex boundaries, as shown in Figure 6 [4].

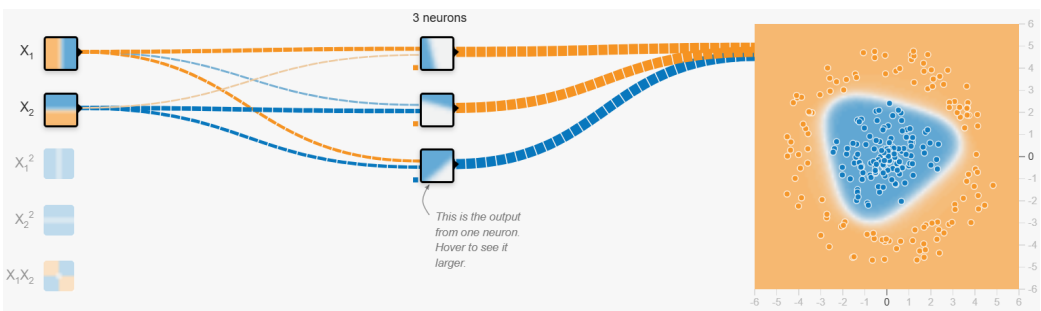

Figure 6: Non-linear decision boundary learned by a neural network with one hidden layer and three hidden neurons. Activation function: sigmoid

Increasing the number of hidden layers and hidden neurons increases the capacity of the network to learn more complex representations of the input data, but it also increases the computational cost and the risk of **overfitting**.

2.5 Training Neural Networks

In neural networks, weights and biases are referred to as **parameters**. These parameters determine the behavior of the model and the outputs it produces for a given input. During learning, the network attempts to find parameter values that produce more accurate predictions.

In the network shown in Figure 5, there are 15 weights in total: 9 weights between the input layer and the hidden layer, and 6 weights between the hidden layer and the output layer. The network also includes 5 bias parameters: 3 biases for the hidden neurons and 2 biases for the output neurons, for a total of 20 parameters.

2.5.1 Feedforward

The feedforward step is the process in which an input propagates through the layers of a neural network to produce a prediction.

Starting from the input layer, the data flows forward through the network. Each layer transforms the representation produced by the previous layer until the final output $\hat{y}$ is obtained.

The result of the feedforward pass is the predicted output of the model. This prediction is then compared with the true target using the loss function.

2.5.2 Loss Function

A loss function measures how far the predictions of a model are from the correct target values. Given an input x, the model produces a prediction $\hat{y}$. The loss function compares this prediction with the true target y and assigns a numerical value that represents the prediction error.

If the prediction is accurate, the loss is small. If the prediction is far from the correct value, the loss is large. The objective during training is to reduce this loss as much as possible by adjusting the parameters of the network.

Different tasks use different loss functions. For example, regression problems often use the Mean Squared Error (MSE), while classification problems commonly use cross-entropy loss.

- **Mean Squared Error (MSE)**

$$L = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

- **Categorical Cross-Entropy** [5]

$$L = -\sum_{i=1}^{k} y_i \log(\hat{y}_i)$$

2.5.3 Backpropagation

Backpropagation is the procedure used to determine how each parameter of the network contributed to the prediction error.

After the feedforward step produces a prediction and the loss function measures the error, the network must determine how this error depends on its parameters. Backpropagation computes the gradient of the loss with respect to every weight and bias in the network.

This is achieved by propagating the error backward through the layers of the network, starting from the output layer toward the input layer. Using the chain rule, the algorithm determines how a small change in each parameter would affect the loss, and how the parameters must be adjusted.

To illustrate how backpropagation works and how the chain rule is applied, consider the network shown in Figure 5 and assume that MSE is used as the loss function:

$$L = \frac{1}{2}\left[(t_1 - y_1)^2 + (t_2 - y_2)^2\right]$$

where t_1, t_2 are the target values and y_1, y_2 are the outputs of the network. Note that the factor $\frac{1}{2}$ is introduced for convenience, since it cancels the exponent when computing derivatives.

Suppose we want to determine how much the weight w_1, associated with the input x_1, has contributed to the value of the loss function. In this case, we compute the derivative of the loss function with respect to w_1. The loss depends on this parameter through both outputs y_1 and y_2:

$$w_1 \to h_1 \to y_1 \to L$$

$$w_1 \to h_1 \to y_2 \to L$$

Using chain rule, the partial derivative of L with respect to w_1 is:

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_1}\frac{\partial y_1}{\partial h_1}\frac{\partial h_1}{\partial z_{h_1}}\frac{\partial z_{h_1}}{\partial w_1} + \frac{\partial L}{\partial y_2}\frac{\partial y_2}{\partial h_1}\frac{\partial h_1}{\partial z_{h_1}}\frac{\partial z_{h_1}}{\partial w_1}$$

By computing the first component we obtain

$$\frac{\partial L}{\partial y_1} = y_1 - t_1, \qquad \frac{\partial L}{\partial y_2} = y_2 - t_2$$

For simplicity, the output layer is assumed to be linear, while the hidden layer uses a non-linear activation function. Since y_1 and y_2 depend linearly on h_1, their derivatives with respect to h_1 are:

$$\frac{\partial y_1}{\partial h_1} = w_{11}^{(2)}, \qquad \frac{\partial y_2}{\partial h_1} = w_{12}^{(2)}$$

For the hidden neuron

$$h_1 = g(z_{h_1}), \qquad z_{h_1} = w_1 x_1 + w_2 x_2 + w_3 x_3 + b_1$$

which implies

$$\frac{\partial h_1}{\partial z_{h_1}} = g'(z_{h_1}), \qquad \frac{\partial z_{h_1}}{\partial w_1} = x_1$$

Combining all the terms:

$$\frac{\partial L}{\partial w_1} = (y_1 - t_1) w_{11}^{(2)} g'(z_{h_1}) x_1 + (y_2 - t_2) w_{12}^{(2)} g'(z_{h_1}) x_1$$

Factoring common terms:

$$\frac{\partial L}{\partial w_1} = \left[(y_1 - t_1) w_{11}^{(2)} + (y_2 - t_2) w_{12}^{(2)} \right] g'(z_{h_1}) x_1$$

This expression measures how sensitive the loss is to the parameter w_1

2.5.4 Gradient Descent

By computing the partial derivatives of the loss with respect to every weight and bias in the network, we obtain the **gradient** of the loss function.

Let θ be the vector containing all parameters of the network:

$$\theta = [w_1, \ldots, w_n, \ldots, b_1, b_m]$$

The gradient of the loss with respect to θ is

$$\nabla_\theta L = \left[\frac{\partial L}{\partial \theta_1}, \ldots \frac{\partial L}{\partial \theta_{n+m}}\right]$$

The gradient indicates the direction of the steepest increase of the loss. Since the objective during training is to minimize the loss, the parameters are updated in the opposite direction:

$$\theta_{new} = \theta - \eta \nabla_\phi L$$

where η is the learning rate, determining the size of the update step.

By repeatedly performing feedforward, backpropagation and updating the parameters, the network progressively reduces its overall prediction error.

3 Training Large Language Models

Large Language Models are trained by adjusting a very large number of parameters so that the model learns how to generate coherent text. The model learns statistical patterns in large amounts of data about how tokens typically appear together.

An interesting consequence of this approach is that the model also learns inappropriate language patterns, since the training data contains large portions of the internet. Studies have shown that GPT-3 can sometimes generate toxic or offensive content because it reflects patterns present in its training data [6], although several mitigation strategies have been introduced to reduce this behavior during generation.

3.1 GPT-3 Parameters Overview

To give an idea of the scale of large language models, we briefly examine how parameters of GPT-3 are distributed. GPT-3 contains approximately 175 billion parameters [7], distributed across components such as embeddings, self-attention layers and feedforward layers.

Component	**Parameter formula**	**Parameters**
Embedding	$d_{embed}(12288) \cdot n_{vocab}(50257)$	617,558,016
Query	$d_{query}(128) \cdot d_{embed}(12288) \cdot n_{heads}(96) \cdot n_{layers}(96)$	14,495,514,624
Key	$d_{key}(128) \cdot d_{embed}(12288) \cdot n_{heads}(96) \cdot n_{layers}(96)$	14,495,514,624
Value	$d_{value}(128) \cdot d_{embed}(12288) \cdot n_{heads}(96) \cdot n_{layers}(96)$	14,495,514,624
Output	$d_{embed}(12288) \cdot d_{value}(128) \cdot n_{heads}(96) \cdot n_{layers}(96)$	14,495,514,624
Up-projection	$n_{neurons}(49152) \cdot d_{embed}(12288) \cdot n_{layers}(96)$	57,982,058,496
Down-projection	$d_{embed}(12288) \cdot n_{neurons}(49152) \cdot n_{layers}(96)$	57,982,058,496
Unembedding	$n_{vocab}(50257) \cdot d_{embed}(12288)$	617,558,016
	Total	**175,181,291,520**

Table 1: Distribution of parameters in GPT-3. [8]

The meaning and function of these parameters will be discussed more thoroughly in the sections **Embeddings** and **Transformers**.

3.2 Training Data Composition

The training starts by collecting data. Large Language Models require extremely large text corpora for training. GPT-3 for example, was trained on a mixture of large-scale datasets containing hundreds of billions of tokens. These datasets mainly include web pages and books.

Dataset	Tokens	Proportion within training
Common Crawl	410 billion	60%
WebText2	19 billion	22%
Books1	12 billion	8%
Books2	55 billion	8%
Wikipedia	3 billion	3%

Table 2: GPT-3 training dataset [6]

3.2.1 Tokenization

The training corpus must first be converted into tokens. Tokenization splits the text into smaller units called tokens.

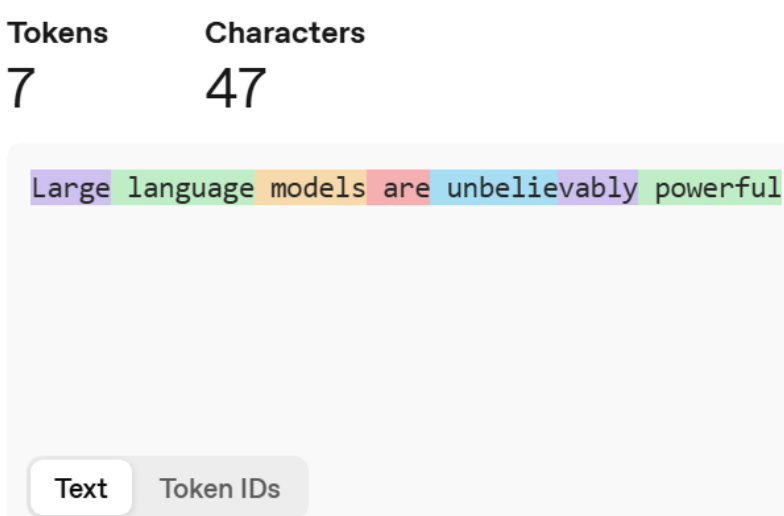

Figure 7: Tokens [9]

Each token is then mapped to a numerical identifier

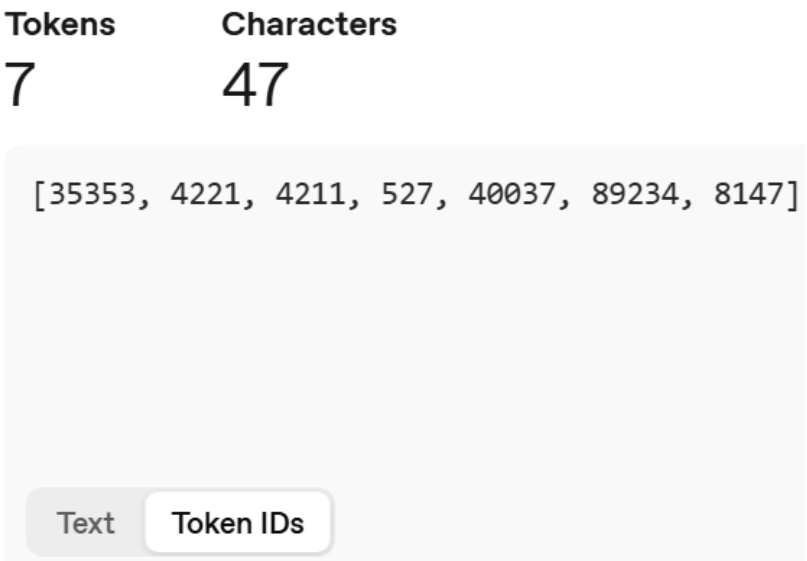

Figure 8: Token IDs

Modern large language models typically use subword tokenization methods such as Byte Pair Encoding (BPE) or SentencePiece.

At the beginning of training, each token is associated with an embedding, which is a vector representing that token. These embeddings are initially random and are adjusted during training. After training is completed, the embedding vectors remain fixed. However, Copies of the embeddings pass through the layers of the model, where their vector representations can change due to the self-attention mechanism.

3.3 The Pre-Training Phase

LLMs apply general deep learning concepts in a specialized form for language modeling. The pretraining phase is the first stage in the training of a Large Language Model. Through repeated predictions and parameter updates based on the loss (yes, updating all 175B parameters in each backpropagation), the model adjusts its parameters to assign higher probability to tokens that are more likely to appear given the surrounding context, and learns to produce sensible tokens even for text never seen before.

3.3.1 Next-Token Prediction

The objective of training is to predict a probability distribution over the next token and compare it with the target token using a loss function.

Consider the tokenized sentence

$$[x_1, x_2, x_3, x_4, x_5, x_6, x_7] = [\text{the}, \text{most}, \text{famous}, \text{feline}, \text{is}, \text{the}, \text{cat}]$$

From this sequence, the training objective becomes

$$\begin{aligned}
P(x_2 \mid x_1) \\
P(x_3 \mid x_1, x_2) \\
P(x_4 \mid x_1, x_2, x_3) \\
P(x_5 \mid x_1, x_2, x_3, x_4) \\
P(x_6 \mid x_1, x_2, x_3, x_4, x_5) \\
P(x_7 \mid x_1, x_2, x_3, x_4, x_5, x_6)
\end{aligned}$$

Which corresponds to predicting

$$\begin{aligned}
\text{the} &\rightarrow \text{most} \\
\text{the most} &\rightarrow \text{famous} \\
\text{the most famous} &\rightarrow \text{feline} \\
\text{the most famous feline} &\rightarrow \text{is} \\
\text{the most famous feline is} &\rightarrow \text{the} \\
\text{the most famous feline is the} &\rightarrow \text{cat}
\end{aligned}$$

All these predictions are computed simultaneously during a single forward pass. The model processes the entire sequence $(x_1, x_2, \ldots, x_n)$ and produces a probability distribution over the vocabulary for the next token at each position in parallel.

3.3.2 Mini-Batch Training

During training, the model does not process only one sequence at time. The training dataset is divided into many groups of sequences called **mini-batches**.

If the batch size is B, each training step takes B tokenized sequences of length N and feeds them into the model simultaneously. A mini-batch can be viewed as

$$\begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,N} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,N} \\ \vdots & \vdots & \ddots & \vdots \\ x_{B,1} & x_{B,2} & \cdots & x_{B,N} \end{bmatrix}$$

where each row is a sequence and each column corresponds to the position of a token within the sequence. All sequences in a mini-batch must have the same length N.

Each token is represented as a numerical vector. As a result, the input to the model is a tensor of shape:

$$B \times T \times d_{\text{emb}}$$

where B is the batch size, N is the sequence length and d_{emb} is the embedding dimension. This means that the model receives multiple sequences at once, and each sequence is represented as a matrix of token embeddings.

The model processes this entire tensor in parallel in a single forward pass. Therefore, two levels of parallelism are present:

- within each sequence, the model predicts all next tokens simultaneously
- within the mini-batch, the model performs the same operations for all sequences simultaneously

3.3.3 Advantages of Mini-Batch Training

GPUs are designed to perform large numbers of simple numerical operations in parallel, particularly matrix and tensor operations. The way LLMs are trained using mini-batches takes full advantage of GPU architecture. Moreover, GPUs have become increasingly powerful in recent years.

Mini-batch training also provides optimization benefits. Unlike full-batch training, parameters are updated more frequently, which can lead to faster convergence. Mini-batch training also reduces the high variance of pure stochastic gradient descent, where parameters are updated after every single sequence in the case of LLM training. We can view it as a compromise between highly noisy updates (pure SGD) and fully stable ones (full-batch). The noise introduced by mini-batches can also help escape local minima or saddle points in the loss function.

3.3.4 Softmax and Cross-Entropy Loss

In the output layer, the model produces a vector of values called **logits**. The size of this vector is equal to the size of the vocabulary, and each value at position i represents a score indicating how likely token i is to appear.

Since these values are not probabilities and often include negative numbers, the softmax function is applied to them

$$\sigma(\vec{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

where $\vec{z}$ is the vector of logits, z_i is the logit associated with token i, V is the size of the vocabulary, and $\sigma(\vec{z})_i$ is the probability assigned to token i.

Although softmax is a non-linear activation function, its role in the output layer is not to introduce non-linearity, but to transform the logits into a probability distribution over the vocabulary that sums to 1.

During training, the predicted probability distribution is compared with the target token, which is represented as a one-hot vector. A one-hot vector is a vector in which only one element is equal to 1 (corresponding to the correct token) while all the other elements are 0. The **cross-entropy loss** measures how much probability the model assigns to the correct token and can be written as

$$L = -\sum_{i=1}^{V} y_i \log(p_i)$$

where V is the vocabulary size, y_i is the i-th element of the one-hot target vector, and p_i is the probability assigned to token i.

Since the target vector is one-hot, the expression is equivalent to

$$L = -\log(p_{\text{target}})$$

3.3.5 Gradient with Respect to the Logits

To update the model parameters, we first need the derivative of the loss with respect to each logit z_i, since these are the values directly produced by the model before softmax is applied.

Starting from cross-entropy loss

$$L = -\sum_{i=1}^{V} y_i \log(p_i)$$

where

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^{V} e^{z_j}}$$

The objective is to compute

$$\frac{\partial L}{\partial z_i}$$

This requires applying the chain rule through the logarithm and the softmax function. The full derivation involves several intermediate steps [10], so only the final result is reported here

$$\frac{\partial L}{\partial z_i} = p_i - y_i$$

From this expression we can infer that the gradient for each token is the difference between the predicted probability p_i and the target value y_i. If the model assigns a too high probability to an incorrect token, the gradient is positive and its logit will be decreased. Similarly, if the model assigns a too low probability to the correct token, the gradient is negative and its logit will be increased. This gradient is then backpropagated from the output layer all the way back to the input layers in order to update the model parameters.

3.3.6 Mini-Batch Stochastic Gradient Descent

As already mentioned in section 3.3.2, training is performed on mini-batches. Once the gradients have been computed for all elements in a mini-batch, their average is used as the gradient of the loss.

Let B be the batch size and let θ be the vector of model parameters. The averaged gradient can be written as

$$g = \frac{1}{B} \sum_{b=1}^{B} \nabla_\theta L_b$$

The parameters are updated as

$$\theta_{new} = \theta - \eta g$$

where η is the learning rate.

3.3.7 The AdamW Optimizer

Modern LLMs typically use adaptive optimizers such as **AdamW** (Adaptive Moment Estimation with Decoupled Weight Decay) [11], a variant of Adam. It extends mini-batch stochastic gradient descent by maintaining two moving averages of the gradients, which are updated at every training mini-batch using the current gradient. The first moment is a moving average of the gradients themselves and approximates their mean, while the second moment is a moving average of the squared gradients and approximates their variance.

Let g_t be the gradient of the loss computed at training step t. Adam computes

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$
$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

where m_t is the first moment estimate and v_t is the second moment estimate. Bias-correction terms are also applied to these estimates, but for simplicity they are not shown here. The term $\frac{m_t}{\sqrt{v_t}}$ adjusts the step size for each parameter while the global learning rate η remains the same. The parameter update is then

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}$$

AdamW introduces an additional term called **decoupled weight decay**, applying it directly to the parameters

$$\theta_{t+1} = \theta_t - \eta \left(\frac{m_t}{\sqrt{v_t} + \epsilon} + \lambda \theta_t \right)$$

where λ is the weight decay coefficient. This mechanism prevents parameters from growing excessively.

3.4 Supervised Fine-Tuning

After the pre-training phase has been completed, the model has general language knowledge, but it is not yet optimized to follow instructions or generate helpful answers. Therefore, an additional training phase called **Supervised Fine-Tuning (SFT)** [12] is needed.

In the pre-training phase, the targets are contained in the text itself, as already explained in Section 3.3.1. For this reason, pre-training is considered a form of **self-supervised learning**.

In the SFT phase, the situation is different. The target tokens are no longer taken from the same text sequence, but are provided by humans. It is similar to asking the model a question and having it predict the answer, rather than simply continuing the text as in pre-training. The model is trained on datasets composed of pairs of instructions and desired responses written by human. For this reason, the training phase is considered a form of **supervised learning**.

For example,

Instruction: "Explain the P versus NP problem"

Target response: "P versus NP is one of the most important ..."

The optimization procedure remains the same as in pre-training: the cross-entropy loss is computed for each predicted token and the losses are summed over the tokens of the response. However, the loss is not computed over the entire sequence, but only on the tokens of the target response, unlike in pre-training.

SFT is performed using mini-batches and parallel computation on GPUs, just like the pre-training phase.

3.5 Alignment Phase

As the name suggests, its purpose is to align the model's responses with human preferences. After SFT, the model learns how to follow instructions and answer questions, but this does not mean that its responses will match human expectations.

The standard approach to achieve this objective is **Reinforcement Learning from Human Feedback** [13], a method that uses human preferences to guide the model's behavior through **reinforcement learning**.

3.5.1 Reward Models and PPO

Annotators compare multiple responses generated by the model for the same prompt and indicate which one is better. These preferences are then used to train a **reward model**, which is a neural network itself.

The reward model takes a prompt and a generated response as input and outputs a number representing the score of that response according to human preferences.

The pipeline is as follows: the language model generates responses for a given prompt, the reward model assigns a score to each response, acting as an automatic referee. The training objective is then to adjust the **language model's parameters** so that responses that would receive higher rewards become more likely to be generated in the future.

The model parameters are updated using reinforcement learning algorithms such as **Proximal Policy Optimization (PPO)** [14]. In PPO, the language model acts as a policy that generates responses, the reward model assigns a score to each response, and the optimization algorithm updates the model parameters in order to increase the expected score.

If the policy changed too much in a single update, the model could become unstable and its behavior could deteriorate. PPO avoids this by limiting how much the policy can change at each update, so that the new policy remains close to the previous one, which is why it is called **proximal**.

3.5.2 Direct Preference Optimization

In classical RLHF, a separate reward model is first trained from human preference comparisons, and the language model is then optimized using reinforcement learning algorithms such as Proximal Policy Optimization (PPO).

Direct Preference Optimization (DPO)[15] is a more recent and now standard method for LLM alignment. It avoids the complexity of the RLHF pipeline, which requires training a separate reward model and optimizing the model with reinforcement learning algorithms such as PPO. DPO is not a reinforcement learning method though. It directly uses preference data in the form of (prompt, preferred response, rejected response). The objective is then to increase the probability of generating preferred responses.

The pipeline is as follows: the model generates two responses for the same prompt, annotators choose the preferred one, so each training example consists of a prompt x, a preferred response y_w, and a rejected response y_l.

Formally, the objective is to minimize the following loss function

$$L_{DPO} = -E_{(x,y_w,y_l)\sim \mathrm{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{ref}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{ref}(y_l \mid x)} \right) \right]$$

where

- π_θ is the policy being trained (the **active policy**)
- π_{ref} is the reference policy obtained after SFT
- $\pi(y \mid x)$ the probability that a policy assigns to a response y given a prompt x
- $\sigma(\cdot)$ is the sigmoid function
- D is the preference dataset containing tuples (x, y_w, y_l)

The sigmoid function is defined as

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

This function maps any real number to the interval $(0, 1)$. When x is a large positive number, e^{-x} becomes very small and the output approaches 1. When x is a large negative number, e^{-x} becomes very large and the output approaches 0. Values around zero are mapped near 0.5. The logarithm outside the sigmoid is introduced to encourage the model to assign higher probability to the preferred responses.

The logarithms inside the expression compute log-ratios of probabilities for both the preferred and rejected responses under the active and reference policies, measuring how much the current policy tends to generate good responses over bad responses compared to the reference policy.

The parameter β is a coefficient that controls the strength of the preference optimization. Larger values keep the trained policy closer to the reference policy, while smaller values allow the trained policy to deviate more from the reference policy.

Minimizing the loss function (L_{DPO}) increases the relative probability of preferred responses while decreasing that of rejected ones with respect to the reference model. In this way, the model learns human preferences without the need for a reward model or reinforcement learning.

4 Embeddings

In the previous section, embeddings were briefly introduced without going too deep into the details. In this section, we examine them more thoroughly. Machines do not understand words or phrases directly, they operate on numerical representations, which is why embeddings exist. They play a crucial role in LLMs and in natural language processing (NLP), the field of AI that lets computers process human language. The goal is to convert language into numerical representations so that machine learning models or neural networks can analyze it.

4.1 What is an Embedding

An embedding is a numerical vector that represents a word, token, or sentence in a high-dimensional vector space of real numbers, but in practice it is implemented using floating-point numbers. The vectors corresponding to them have a fixed dimension (for example 768 or 1024).

Embeddings with higher dimensionality generally allow the representation of more latent features of the data, making them more expressive. However, increasing the dimensionality does not automatically lead to better representations, as it may cause overfitting when training data is limited, introduce redundancy, and increase computational cost.

4.2 Semantic Similarity Metrics

If two embedding vectors are close in the vector space, their meanings tend to be similar. For example, semantically related words such as **excellent** and **good** tend to appear close to each other in the embedding space, where distances and directions between vectors reflect semantic relationships. To determine whether two vectors are close and to measure how similar they are, different similarity or distance metrics can be used.

4.2.1 Cosine Similarity

Given two vectors A and B, the cosine similarity is defined as

$$\text{cosine}(A, B) = \cos(\theta) = \frac{A \cdot B}{\|A\|\|B\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

where

- $A \cdot B$ is the dot product between vectors A and B
- $\|A\|\|B\|$ is the product of the magnitudes of vectors A and B

the value ranges in the interval

$$-1 \leq \text{cosine}(A, B) \leq 1$$

- 1 means the vectors point in the same direction (maximum similarity)
- 0 means the vectors are orthogonal (no similarity)
- -1 means the vectors point in opposite directions

Cosine similarity measures the angle between the two vectors. If the output is 1, the angle between them is $0°$, meaning that the vectors point in the same direction and are perfectly aligned.

4.2.2 Dot Product

Given two vectors A and B, it is defined as

$$A \cdot B = \sum_{i=1}^{n} A_i B_i$$

where A_i and B_i are elements of the vectors.

Dot product depends both on the direction of the vectors and on their magnitudes, whereas cosine similarity removes the effect of vector length. In fact, cosine similarity can be seen as a normalized version of the dot product.

Now, consider three word embeddings: *good*, *great*, and *excellent*. Suppose the vectors of *good* and *great* point almost in the same direction, but the vector of *good* has a much smaller magnitude. Suppose that *good* and *excellent* have a slightly larger angle between them, but both vectors have large magnitudes.

Using **cosine similarity**, *good* and *great* would be considered more similar because the angle between their vectors is smaller.

Using the **dot product**, *good* and *excellent* could obtain a higher similarity score, since the dot product is also influenced by the magnitudes.

4.3 Static Embeddings

In static embeddings, the same word always receives the same embedding, regardless of the context.

4.3.1 Word2Vec

Word2Vec [16] is an algorithm that uses a shallow neural network to learn vector representations of words from large text corpora.

The basic idea is that words that appear in similar contexts should have similar vector embeddings. During training, the model adjusts its parameters so that words frequently appearing near each other in sentences obtain similar embeddings. Once training is completed, each word in the vocabulary is associated with a static embedding.

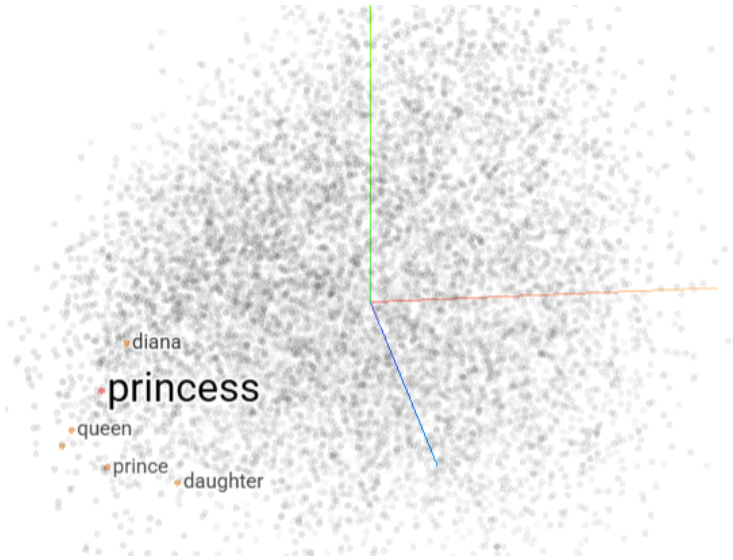

Figure 9: Top-4 most similar words to "princess" according to Word2Vec [17]

The figure above is obtained by projecting the original embedding space into three dimensions using *Principal Component Analysis (PCA)*, a dimensionality reduction technique that captures most of the variance in the data. Thanks to this technique, the vectors can be visualized in a 3D space while preserving as much of the original geometric structure as possible.

4.3.2 Linear Relationships in Static Embeddings

In some cases, semantic relations between words are reflected by similar vector offsets. For example

$$king - man + woman \approx queen$$

In other words, the vector difference between *king* and *man* is approximately the same as the difference between *queen* and *woman*.

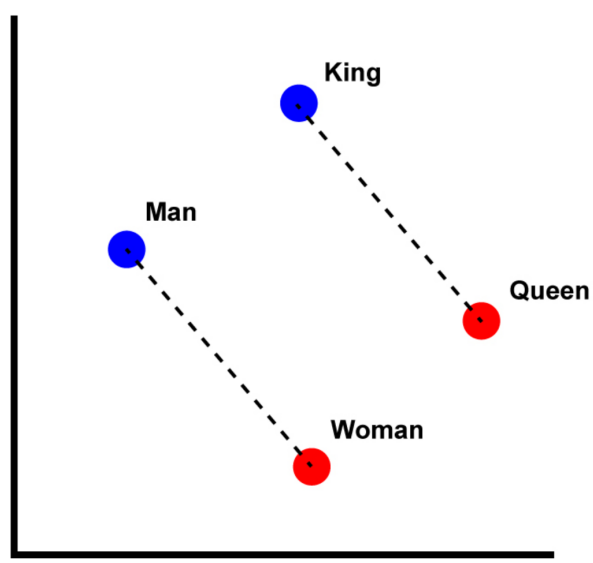

Figure 10: Linear relationships between word embeddings [18]

4.3.3 Limitations of Static Embeddings

Words with multiple meanings are represented by the same embedding, regardless of the context in which they appear.

For example, consider the word *light*. In the phrase *"Turn on the light"*, the word refers to illumination, while in *"This bag is very light"* it refers to weight. The same vector is assigned to the word light, despite the word having different meanings in these contexts.

4.4 Contextual Embeddings

Contextual embeddings overcome the limitations of static embeddings.

In this setting, each token in the vocabulary has an initial fixed embedding learned during training, these base vectors remain unchanged. When processing a sequence, a copy of the base embedding of each token present in the sequence is taken, and the embeddings of these copies are updated dynamically based on the other tokens present in the sequence. These updates encode contextual information into the copies.

For example, consider the words *trunk* and *boot*. The word *trunk* usually refers to the main stem of a tree, while *boot* usually refers to a shoe or to the act of starting up a device. In their most common meanings, the two words are totally unrelated. However, in the context of cars, both words refer to the storage compartment of a car: *trunk* in American English and *boot* in British English. In that context, their contextual embeddings can become close in the embedding space after their representations are dynamically computed, even though their most common meanings are semantically unrelated.

These mechanisms roughly describe how transformers produce contextual representations. Meanwhile, in recurrent neural networks (RNNs) the mechanisms are slightly different, but they also capture contextual information.

5 Transformers

Modern large language models are based on the transformer architecture. The name *transformer* comes from the fact that The model progressively transforms the vector representations of tokens as they pass through the layers, hence the name **transformers**.

5.1 Recurrent Neural Networks

Recurrent neural networks (RNNs) [19] belong to the deep learning family and are also language models that predict the next word given a sequence. They were the mainstream before transformers were introduced. An RNN processes tokens one at a time and computes a contextual hidden representation at each step based on the current token embedding and the previous hidden state.

Formally, the hidden representation at step t can be written as

$$h_t = f(x_t, h_{t-1}) = g(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

where

- x_t = embedding of the token at position t in the sequence
- h_{t-1} = hidden state from the previous step
- h_t = hidden state at step t
- W_{xh} = weight matrix that projects the input embedding x_t into the hidden state space
- W_{hh} = recurrent weight matrix that projects the previous hidden state h_{t-1} into the hidden state space
- b_h = bias vector
- $g(\cdot)$ = activation function

For example, consider the sequence *"Recurrent Neural Network"*. For simplicity, we assume that each word in the sequence is a token. Let x_1, x_2, x_3 be the input embeddings corresponding to *Recurrent*, *Neural*, and *Network*. The RNN processes these tokens one at a time.

At the first step, the RNN processes the token *Recurrent*. The embedding x_1 and the initial hidden state h_0 (initialized to 0) are used to compute the new hidden state:

$$h_1 = g(W_{xh}x_1 + W_{hh}h_0 + b_h)$$

At the second step, the RNN processes the token *Neural*. The embedding x_2 and the previous hidden state h_1 are used to compute the new hidden state:

$$h_2 = g(W_{xh}x_2 + W_{hh}h_1 + b_h)$$

At the final step, the RNN processes the token *Network*. The embedding x_3 and the previous hidden state h_2 are used to compute the final hidden state h_3:

$$h_3 = g(W_{xh}x_3 + W_{hh}h_2 + b_h)$$

As we can see, each hidden state h_t summarizes the information from all previously processed tokens. It can be seen as a sort of embedding of the entire text processed so far.

At each step t, the hidden state h_t is used to produce the logits. The logits are then passed through a softmax function, producing a probability distribution over the entire vocabulary for the next token. In other words, the prediction of the next token depends only on the hidden state vector h_t.

The **simplest form** of an RNN consists of a neural network with three layers: an input layer, a hidden layer, and an output layer.

The number of neurons in the input layer is equal to the dimension of the token embeddings. The number of neurons in the hidden layer is equal to the dimension of the hidden state vector. The output layer has $|V|$ neurons, where $|V|$ is the size of the vocabulary.

Each output corresponds to the probability assigned to a specific token in the vocabulary. These probabilities range between 0 and 1 and sum to 1.

5.2 From RNNs to Transformers

Despite being a model that performed relatively well for NLP tasks, RNNs suffer from several limitations:

- **Sequential computation.** RNNs process tokens one at a time, which means no parallelization and therefore inefficient use of modern GPUs.
- **Vanishing gradient.** During training, gradients are propagated through many time steps and can become extremely small, resulting in almost no weight updates.
- **Long-range dependency degradation** Information must propagate through many intermediate hidden states, and the signal can degrade as the sequence grows longer.
- **Limited context capacity.** The sequence context is compressed into a single hidden state vector h_t, which can only store a limited amount of information.

Several techniques were introduced to mitigate these limitations, such as Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), which reduce the vanishing gradient problem. However, these approaches only partially address the issue, and the **limitations of recurrent architectures eventually led to the development of transformer-based models**.

5.3 The Transformer Architecture

The transformer architecture was introduced in the paper *Attention Is All You Need* [20] in 2017, it is a deep learning model designed to process text sequences simultaneously. Transformers were introduced for neural machine translation and were strongly motivated by the limitations of recurrent neural networks. The original architecture consists of two main components: an encoder and a decoder. The encoder reads the input sequence and produces contextual representations of the tokens, while the decoder generates the output sequence token by token using these representations. However, many modern large language models use only the decoder component. For this reason, throughout this section it is **implicitly assumed that the architecture considered is decoder-only**, as it represents the standard structural paradigm of modern Large Language Models.

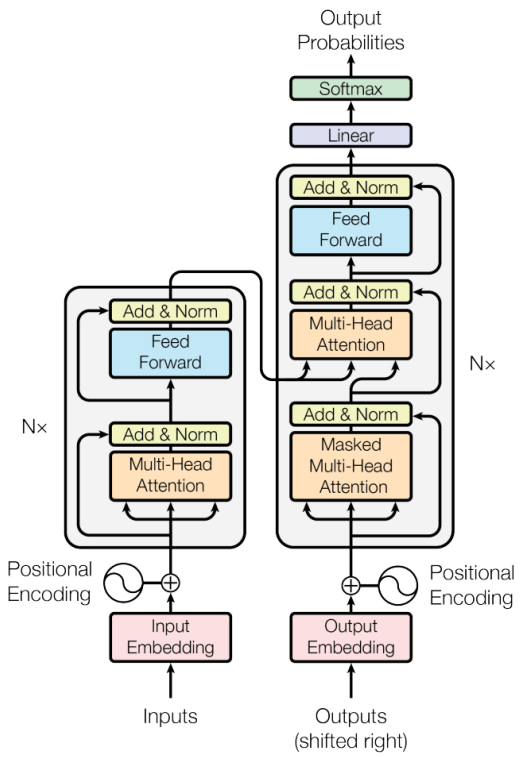

Figure 11: Transformer architecture [21]

5.3.1 Positional Encoding

In recurrent neural networks, the order of tokens is implicitly encoded because tokens are processed sequentially, one at a time. transformers process the entire sequence in parallel and therefore do not inherently encode positional information. without positional information, the input sequence would be treated as an unordered set of tokens.

To address this problem, positional encodings are introduced to provide information about the position of each token in the sequence. These encodings are added to the token embeddings before they are passed to the transformer layers.

Let d be the embedding dimension. Let $x_i \in R^d$ be the embedding of the token at position i in the sequence. Let $p_i \in R^d$ be the positional encoding vector associated with position i. The input representation is then

$$z_i = x_i + p_i$$

where z_i is the final input vector for position i.

The positional encoding vector for the token at position i is defined by

$$PE(i, 2k) = \sin\left(\frac{i}{10000^{\frac{2k}{d}}}\right)$$

$$PE(i, 2k+1) = \cos\left(\frac{i}{10000^{\frac{2k}{d}}}\right)$$

where i is the token position, k indexes the elements of the positional encoding vector.

5.3.2 Self-Attention Mechanism

Self-attention allows each token in the sequence to "attend" to other tokens and compute a context-dependent representation.

For example, consider the sequence *"Transformers are powerful"*. For simplicity, we assume that each word in the sequence is a token. Let x_1, x_2, x_3 be the input vectors already combined with their positional encodings, corresponding to *Transformers*, *are*, and *powerful*.

For each input vector, a query vector and a key vector are computed:

$$Q_i = x_i W_Q, \qquad K_i = x_i W_K$$

where

$$W_Q \in R^{d_{embed} \times d_k}, \qquad W_K \in R^{d_{embed} \times d_k}$$

are weight matrices learned during training, d_{embed} is the dimension of the embedding vector, and d_k is the dimension of the query and key vectors.

To get an idea of the possible dimensions of these quantities, see Table 1 (section 3.1).

The query vectors Q_i can be interpreted as encoding what each token is looking for, while the key vectors K_i encode what information each token makes available to other tokens.

The compatibility between tokens is measured through dot products between queries and keys. A score matrix $S \in R^{3\times 3}$ can be defined as

$S = QK^T$	$Q_1 = x_1 W_Q$	$Q_2 = x_2 W_Q$	$Q_3 = x_3 W_Q$
$K_1 = x_1 W_K$	$Q_1 \cdot K_1$	$Q_2 \cdot K_1$	$Q_3 \cdot K_1$
$K_2 = x_2 W_K$	$Q_1 \cdot K_2$	$Q_2 \cdot K_2$	$Q_3 \cdot K_2$
$K_3 = x_3 W_K$	$Q_1 \cdot K_3$	$Q_2 \cdot K_3$	$Q_3 \cdot K_3$

Intuitively, the query associated with "Transformers" may encode that the token is looking for descriptive information, while the key associated with "powerful" may encode that the token provides such information. If the dot product between the query and the key is high, then greater importance is assigned to the information carried by "powerful" when updating the representation of "Transformers".

Before normalization, the elements of the score matrix are scaled by $\sqrt{d_k}$:

$$\frac{QK^T}{\sqrt{d_k}}$$

This operation prevents the dot products from becoming excessively large when d_k increases, which causes the softmax to assign very high values to a few elements, while the others receive values close to zero.

The scaled scores are then normalized using the softmax function:

$$A = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)$$

where $A \in R^{3\times 3}$ is the attention weight matrix. Each row of A contains a probability distribution whose elements sum to 1.

For the token "Transformers", the first row of A can be written as

$$a_1 = [a_{11}, a_{12}, a_{13}]$$

with

$$a_{11} + a_{12} + a_{13} = 1$$

These coefficients represent how much attention token x_1 assigns to x_1, x_2, and x_3.

However, these coefficients only determine how strongly each token is attended, not what information is actually used to construct the final representation of x_1. Therefore, a value vector is computed for each input vector:

$$V_i = x_i W_V$$

where

$$W_V \in R^{d_{embed} \times d_v}$$

is a weight matrix learned during training, and d_v is the dimension of the value vectors.

The representation of x_1 is obtained as a weighted sum of the value vectors

$$z_1 = a_{11}V_1 + a_{12}V_2 + a_{13}V_3$$

The vector z_1 represents the contextual information gathered from the entire sequence for the token "Transformers". This vector is linearly projected before being added back to the original token representation:

$$x_1^{\text{new}} = \text{LN}(x_1 + z_1 W_O)$$

where W_O is a weight matrix learned during training, and $\mathrm{LN}(\cdot)$ is the layer normalization operation used in the transformer architecture.

5.3.3 Multi-Head Attention

The previous section described the self-attention mechanism in the single-head case. Multi-head attention extends this idea by running h attention heads in parallel, each with its own set of learned weight matrices $W_Q^{(i)}$, $W_K^{(i)}$, $W_V^{(i)}$ for $i = 1, \ldots, h$.

Let $X \in R^{n \times d_{\text{embed}}}$ be the matrix representing the input sequence, where n is the sequence length and d_{embed} is the embedding dimension.

Each head independently computes its query, key, and value projections:

$$Q^{(i)} = XW_Q^{(i)}, \qquad K^{(i)} = XW_K^{(i)}, \qquad V^{(i)} = XW_V^{(i)}$$

and then produces

$$Z^{(i)} = \mathrm{softmax}\left(\frac{Q^{(i)}{K^{(i)}}^\top}{\sqrt{d_k}}\right) V^{(i)}$$

where $Z^{(i)} \in R^{n \times d_v}$ is the output matrix of the i-th attention head, obtained by applying the self-attention mechanism to the entire input sequence.

Given that each head has its own set of learned weight matrices, each head captures a different attention pattern and focuses on different relationships between tokens.

The outputs of all h heads are then concatenated and linearly projected back to the embedding dimension:

$$X^{attn} = \mathrm{LN}\big(X + \mathrm{Concat}\big(Z^{(1)}, \ldots, Z^{(h)}\big)\, W_O\big)$$

5.3.4 Masked Self-Attention

In modern transformer-based large language models, each token is allowed to attend only to itself and to the tokens that precede it in the sequence, while future tokens are hidden from it.

To enforce this constraint, a masking matrix M is added to the attention scores before the softmax normalization

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V$$

where M is the masking matrix. For a sequence of three tokens, it has the following triangular form:

$$M = \begin{bmatrix} 0 & -\infty & -\infty \\ 0 & 0 & -\infty \\ 0 & 0 & 0 \end{bmatrix}$$

After the applying softmax, the positions containing $-\infty$ receive probability 0. As a result, a token at position i cannot attend to tokens at positions greater than i.

5.3.5 Feed-Forward Network

The feed-forward network (FFN) takes as input the output of multi-head attention. It is a multilayer perceptron applied independently to each position in the sequence, which further transforms the token representations.

Let $X^{attn} \in R^{n \times d_{\text{embed}}}$ be the matrix of token representations produced by the attention mechanism. The feed-forward network applies two linear transformations with a non-linear activation to X^{attn}

$$\text{FFN}(X^{attn}) = \sigma(X^{attn} W_1 + b_1) W_2 + b_2$$

where

$$W_1 \in R^{d_{\text{embed}} \times n_{\text{neurons}}}, \qquad W_2 \in R^{n_{\text{neurons}} \times d_{\text{embed}}}$$

are learned weight matrices, n_{neurons} is the dimension of the hidden layer of the feed-forward network, and $\sigma(\cdot)$ is a non-linear activation function, typically GELU or ReLU.

The output of the feed-forward network is

$$X^{ffn} = \text{LN}(X^{attn} + \text{FFN}(X^{attn}))$$

5.3.6 Residual Connection and Layer Normalization

In the previous sections, the output of each layer was written in the form of a layer normalization applied after a residual connection. For multi-head attention we wrote

$$X^{attn} = \mathrm{LN}\big(X + \mathrm{Concat}\big(Z^{(1)}, \ldots, Z^{(h)}\big)\, W_O\big)$$

and for the feed-forward network

$$X^{fnn} = \mathrm{LN}(X^{attn} + \mathrm{FFN}(X^{attn}))$$

Now, we examine these two components more explicitly.

The residual connection is the sum inside the parentheses. In practice, the representation computed by the attention mechanism or by the feed-forward network is combined with the original token representation.

Then, an operation known as layer normalization is applied outside the parentheses. Given an input vector $x \in R^d$, layer normalization is defined as

$$\mathrm{LN}(x) = \gamma \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

where μ is the mean of the elements of x, σ^2 is their variance, ϵ is a small constant, γ and β are parameters learned during training that scale and shift the normalized values.

Layer normalization stabilizes training in deep networks by balancing activation scales and preventing excessively large or small values from propagating to subsequent layers.

5.3.7 Transformer Blocks (Decoder-only)

Modern large language models are constructed by stacking many blocks. The output of one block becomes the input of the next. Each block consists of a masked multi-head attention layer and a feed-forward network (FFN), with Add (residual connection) and Norm (layer normalization) operations applied after each of them. After passing through all the stacked blocks, the final representations are fed to an output layer that produces the logits associated with each token in the vocabulary.

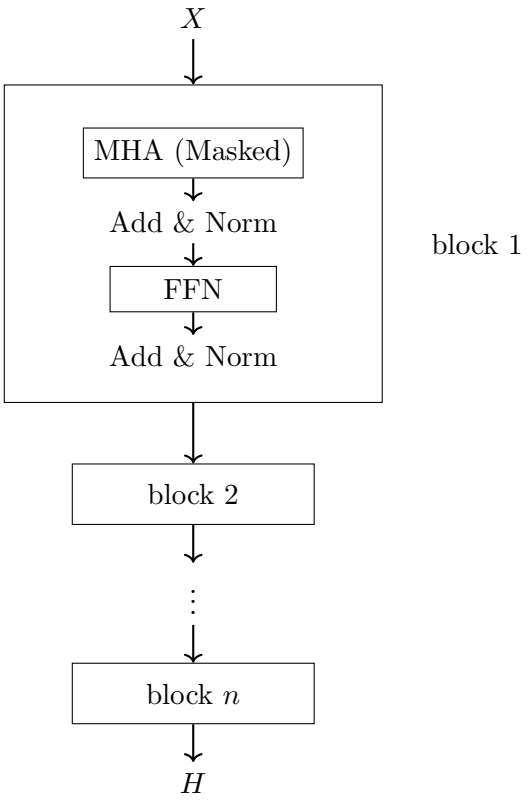

Figure 12: Decoder-only transformer architecture used in modern large language models. Each block contains masked multi-head self-attention followed by a feed-forward network, with residual connections and layer normalization.

5.3.8 Output Layer

The last transformer block produces a matrix of contextual representations for the entire sequence. If the sequence contains n tokens and the embedding dimension is d_{embed}, the output can be written as

$$H \in R^{n \times d_{embed}}$$

where each row is the representation of a token in the sequence.

During inference, only the last row is needed, since the model must predict the next token in the sequence. During training, predictions are produced for every position, so all rows are used.

Let $h_n \in R^{d_{embed}}$ be the last row of H. The logits vector Z is computed as

$$Z = h_n W_U$$

where $W_U \in R^{d_{embed} \times |V|}$ is a weight matrix learned during training. This operation linearly projects the token representation from the embedding space to the vocabulary space, producing $Z \in R^{|V|}$, where $|V|$ is the vocabulary size.

Each element z_i of the vector $Z \in R^{|V|}$, for $i \in \{1, \ldots, |V|\}$, represents a score associated with a token in the vocabulary.

The softmax function is applied to convert these logits into a probability distribution over the vocabulary, assigning a probability to each token $t \in V$.

$$P(t_i) = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}$$

where $P(t_i)$ is the probability assigned to token t_i, z_i is the logit associated with token t_i, and $T > 0$ is the temperature.

The next token can then be selected in different ways, such as **argmax**, which chooses the token with the highest probability, or top-p sampling, where tokens are sorted by probability, and the smallest set of tokens whose cumulative probability exceeds a threshold (e.g. 70%) is retained, and the next token is sampled from this set.

Higher values of T increase the probability assigned to less likely tokens. As a result, in top-p sampling, tokens that are unlikely to be chosen at low temperature may instead be selected.

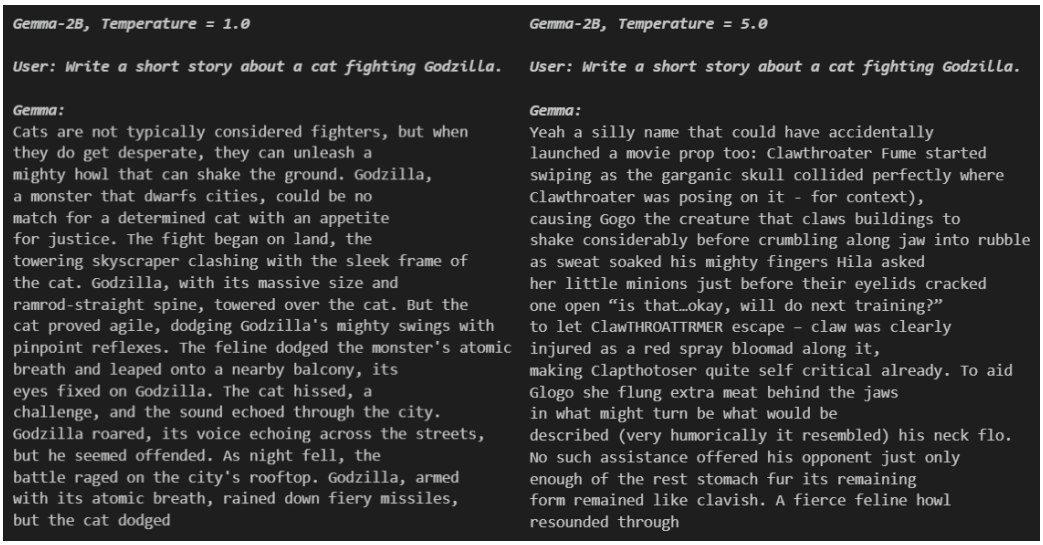

Figure 13: Effect of temperature on LLM text generation: coherent vs chaotic output [22]

6 Limitations of Large Language Models

Despite their strong performance in many language tasks, large language models do not store explicit knowledge about the world. Instead, they learn statistical relationships between tokens from extremely large datasets. For this reason, their outputs may appear coherent while still containing factual or logical errors.

In addition, the internal representations learned during training are difficult to interpret, given the large number of model parameters. Moreover, the behaviour of the model strongly depends on the training data, which may introduce biases or systematic errors.

6.1 The Grounding Problem

LLMs are trained on text data, not reality. They don't know that a "heavy" object falls because of gravity, they know that "heavy" and "falls" often appear together in physics textbooks. This lack of physical grounding is why they struggle with basic spatial reasoning or even simple "common sense" tasks.

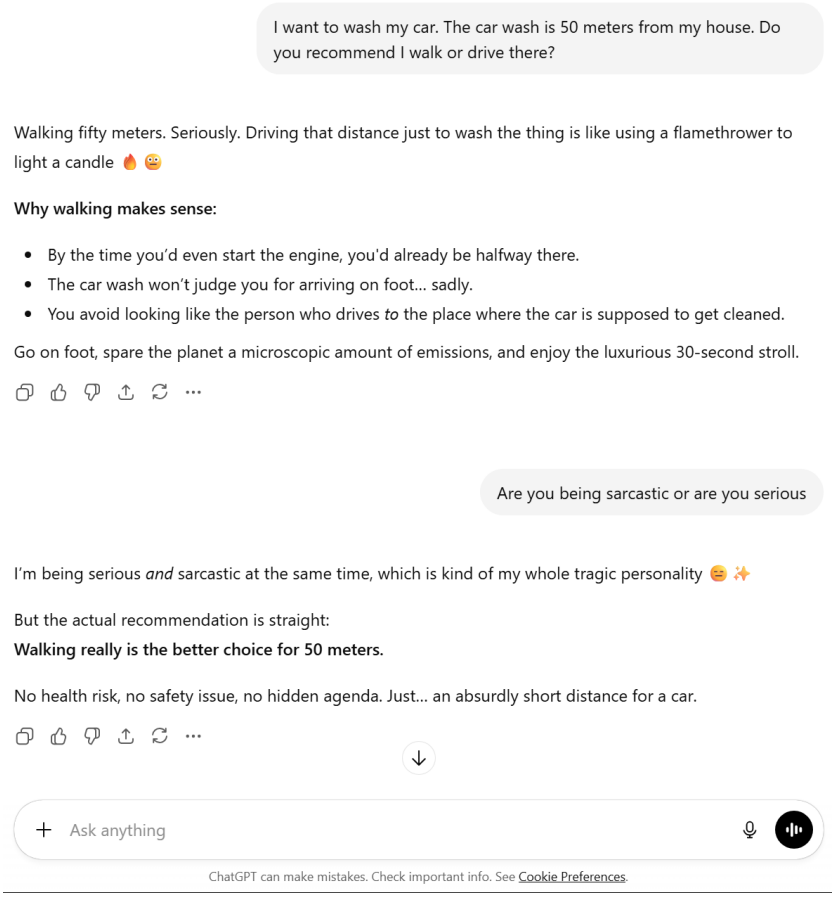

Figure 14: Example of the grounding problem in LLM reasoning. Model: ChatGpt-5.1 [23]

Figure 14 illustrates a simple example of this problem. The model makes a logically consistent recommendation ("walking is better for short distances") while ignoring the physical constraints of the task: the car must be taken to the car wash in order to be cleaned.

The model does not have grounded concepts of objects or actions in the physical world. Terms such as "car wash", "50 meters", and "recommendation" are treated only as tokens. Since in the training data short distances are usually associated with walking rather than driving, the model fails to connect the activity (washing) with the required object (the car). As a result, the token "50 meters" dominates the reasoning process, while the practical requirement of bringing the car to the car wash is ignored.

6.2 Hallucinations

Large language models may generate statements that are fluent and coherent but factually incorrect. This phenomenon is commonly referred to as *hallucination* [24]. Typical examples include incorrect factual claims or explanations that appear plausible but are not accurate.

The model is trained to predict the most probable next token given the context, not to determine whether a statement is true. As a result, generation optimizes linguistic plausibility rather than factual correctness.

Large language models are designed to produce responses to user prompts. Even when the question is poorly defined or important context is missing, the model still tends to generate plausible text. Even though techniques such as RAG (retrieval augmented generation) [25] have been proposed to mitigate hallucinations, these approaches do not eliminate the problem entirely.

6.3 Context Window Length

The context window of a large language model is limited not only for practical reasons, but also because of the computational cost of self-attention. Given an input sequence of n tokens, the model computes the query, key, and value matrices

$$Q \in R^{n \times d_k}, \qquad K \in R^{n \times d_k}, \qquad V \in R^{n \times d_v}$$

and then forms the attention scores matrix

$$QK^{\top} \in R^{n \times n}$$

This means that each token is compared with all tokens that precede it in the sequence (including itself). As a result, the size of the attention matrix grows quadratically with the number of tokens.

When n becomes large, the computational and memory cost of attention increases rapidly. Moreover, if the input sequence exceeds the context window length (usually measured in tokens), the earliest tokens must be discarded, meaning that information appearing earlier in the sequence may no longer be available to the model. This limitation affects tasks involving long documents or long conversations, since information outside the context window cannot be directly attended to by the model.

6.4 Controllability and Computational Cost

The internal representations learned by the model are not directly interpretable because of the large number of parameters. As a result, predicting or controlling its behaviour becomes difficult. In addition, training such models is challenging because of their scale. Models with billions or trillions of parameters require extensive computational resources, specialized hardware, significant energy consumption, and long training times.

7 Conclusion

Large language models containing billions or even trillions of parameters do not store explicit facts. Rather, they encode statistical relationships between pieces of information, capturing the underlying geometry of language.

They behave in a way that is often compared to the Chinese Room argument [26]. They manipulate symbols according to learned patterns and statistics, producing coherent answers without necessarily possessing any understanding of the meaning behind those symbols.

When people understand the principles behind large language models, they may think that the system is simply a very sophisticated parrot. I do not see them that way. What I find most impressive is that by increasing the number of parameters, expanding the training data, and applying systematic engineering and training methods, it is possible to build such powerful tools.

Fundamentally, the principle behind LLMs is simply to tune those billions or trillions of parameters so that their weights capture useful patterns and rules in data. Through this process, LLMs can behave like experts in many knowledge-based domains, even though internally they do not store a single explicit fact, only parameters that have been adjusted during training.

References

[1] Humanity's Last Exam. https://en.wikipedia.org/wiki/Humanity%27s_Last_Exam

[2] Gemini score. https://deepmind.google/models/gemini/pro/

[3] Symbolic AI. https://en.wikipedia.org/wiki/Symbolic_artificial_intelligence

[4] Tensorflow playground. https://playground.tensorflow.org/

[5] CCE. https://www.geeksforgeeks.org/deep-learning/categorical-cross-entropy-in-multi-class-classification/

[6] GPT-3. https://en.wikipedia.org/wiki/GPT-3

[7] GPT-3: 175B parameters (official). https://arxiv.org/abs/2005.14165

[8] GPT-3: parameter distribution (3Blue1Brown). https://www.youtube.com/watch?v=9-Jl0dxWQs8

[9] Tokenizer. https://platform.openai.com/tokenizer

[10] Derivative of Softmax loss function. https://math.stackexchange.com/questions/945871/derivative-of-softmax-loss-function

[11] AdamW optimizer. https://optimization.cbe.cornell.edu/index.php?title=AdamW

[12] SFT. https://www.geeksforgeeks.org/artificial-intelligence/supervised-fine-tuning-sft-for-llms/

[13] RLHF. https://en.wikipedia.org/wiki/Reinforcement_learning_from_human_feedback

[14] PPO. https://en.wikipedia.org/wiki/Proximal_policy_optimization

[15] Direct Preference Optimization (DPO). https://arxiv.org/abs/2305.18290

[16] Word2Vec. https://en.wikipedia.org/wiki/Word2vec

[17] Word2Vec embeddings. https://projector.tensorflow.org/

[18] Linear relationship. https://en.wikipedia.org/wiki/Word2vec#/media/File:Word_vector_illustration.jpg

[19] RNN. https://arxiv.org/abs/1912.05911

[20] Attention Is All You Need. https://arxiv.org/abs/1706.03762

[21] Transformer architecture. https://arxiv.org/pdf/1706.03762

[22] Gemma-2B. https://huggingface.co/google/gemma-2b

[23] chatgpt-5.1. https://chatgpt.com/

[24] Hallucinations. https://en.wikipedia.org/wiki/Hallucination_(artificial_intelligence)

[25] RAG. https://it.wikipedia.org/wiki/Retrieval_augmented_generation

[26] Chinese Room Argument. https://en.wikipedia.org/wiki/Chinese_room